BÁO CÁO THỰC HÀNH LAB F5

Danh sách & lưới trong Flutter — Ứng dụng "Sản phẩm VHU"

ListView.builder · ListView.separated · GridView.builder · Card + ListTile · Dismissible · NC1 + NC2 + NC3 + NC4

Họ và tên: Nguyễn Việt Quang
MSSV: 231A290134
Lớp: CNTT – LTDD

Flutter 3.47.3 (stable) | Dart 3.13.3 | DevTools 2.60.0

Thiết bị kiểm thử: Pixel 6 API 34 (Android 14, x86_64) — 1080 × 2400, mật độ 420 dpi (dpr 2,625)

Project: E:\LTDiDong\F5_231A290134 — package: vn.edu.vhu.ltdd.f5_231a290134

Link GitHub: https://github.com/nvquangggit-sys/F5_231A290134

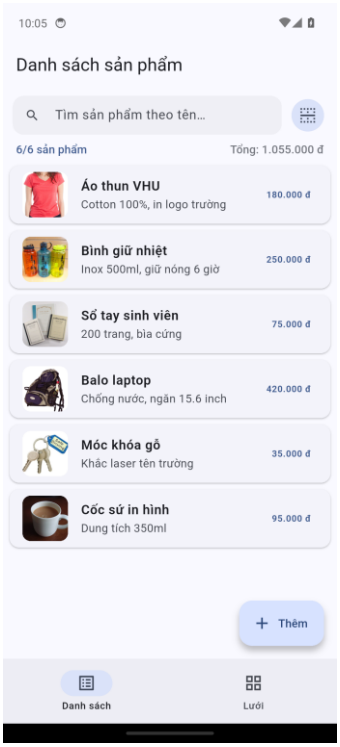
Checkpoint

Checkpoint — sản phẩm bắt buộc

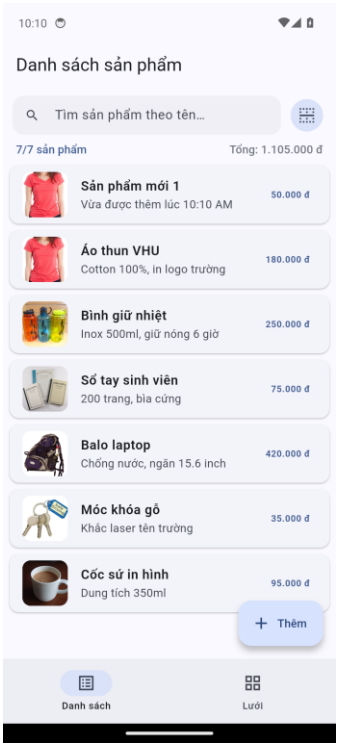
Chạy app: tab Danh sách hiện 6 sản phẩm đủ ảnh / tên / mô tả / giá đã định dạng; bấm + thì mục mới lên đầu danh sách; vuốt sang trái xóa được (có nền đỏ và biểu tượng thùng rác); chuyển tab Lưới thấy các ô đều nhau và tự đổi số cột khi xoay ngang.

Kết quả thật đã kiểm chứng trên máy ảo

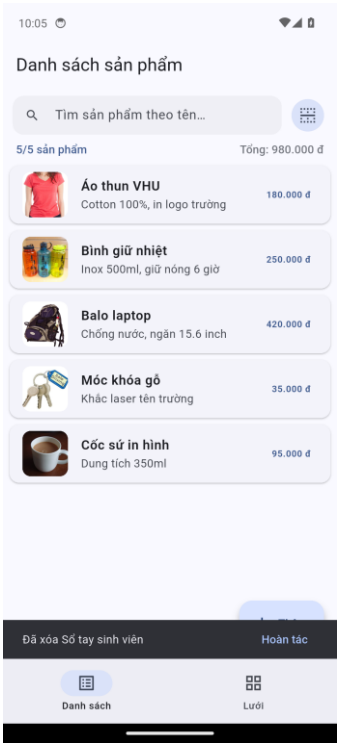
6/6 sản phẩm ở tab Danh sách → bấm Thêm thành 7/7 (mục mới nằm đầu danh sách) → thêm tiếp thành 12/12 để thử cuộn → vuốt xóa đúng "Sổ tay sinh viên" ở giữa danh sách, SnackBar hiện "Đã xóa …" kèm nút Hoàn tác và khôi phục lại 6/6 → xóa hết thì hiện "Danh sách trống" thay vì màn hình trắng → lọc "balo" còn 1/12 → bật đường kẻ thấy ListView.separated → tab Lưới 2 cột (dọc) và 4 cột (ngang), ảnh không méo, không tràn ô. flutter analyze: No issues found! — flutter test: 5/5 pass.



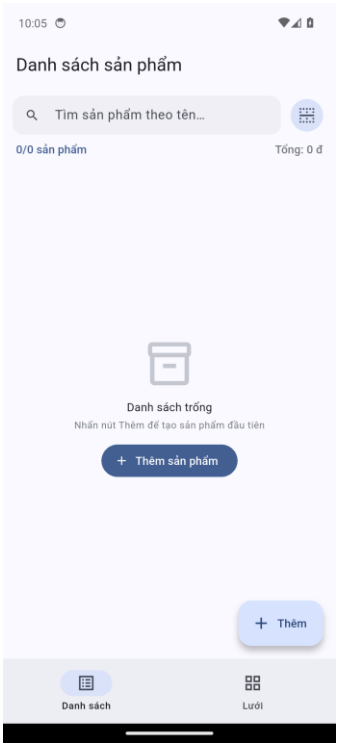
(a) Tab Danh sách — 6 sản phẩm đủ ảnh, tên, mô tả, giá



(b) Bấm + — "Sản phẩm mới 1" chèn lên đầu, 7/7 sản phẩm



(c) Vuốt xóa "Sổ tay sinh viên" — SnackBar kèm nút Hoàn tác



(d) Xóa hết — hiện "Danh sách trống" thay vì màn hình trắng

Hình 1. Bốn trạng thái chính của app F5_231A290134 trên Pixel 6 API 34.

Phần 0. Môi trường, cách chạy và sản phẩm nộp

Trường	Giá trị
Template	flutter create --org vn.edu.vhu.ltdd --project-name f5_231a290134 --platforms android
Tên project / thư mục	F5_231A290134
package (Dart)	f5_231a290134
applicationId (Android)	vn.edu.vhu.ltdd.f5_231a290134
Phiên bản Flutter / Dart	3.47.3 (stable) / 3.13.3
Nền tảng	Android (Material 3, ColorScheme.fromSeed #0468D7)
Kiểm thử	Pixel 6 API 34 — Android 14, x86_64, 1080 × 2400, 420 dpi
Lệnh chạy	flutter pub get → flutter run (hoặc flutter build apk --debug)

Cấu trúc mã nguồn được tách theo trách nhiệm để đọc và bảo trì dễ hơn: dữ liệu ở thư mục data, model ở models, widget dùng chung ở widgets, màn hình ở screens. Tổng cộng 748 dòng Dart.

Tệp	Số dòng	Nhiệm vụ
lib/main.dart	68	MaterialApp + HomePage (AppBar, IndexedStack 2 tab, NavigationBar)
lib/models/san_pham.dart	28	Lớp dữ liệu SanPham (id, tên, mô tả, giá, ảnh) — tách khỏi giao diện
lib/data/kho_san_pham.dart	68	duLieuMau 6 sản phẩm + dinhDangGia() + tongTien()
lib/widgets/anh_san_pham.dart	28	Image.asset dùng chung, có errorBuilder
lib/screens/danh_sach_tab.dart	333	ListView.builder / separated, lọc, thêm, xóa, hoàn tác
lib/screens/album_tab.dart	123	GridView.builder + SliverGridDelegateWithMaxCrossAxisExtent
lib/screens/chi_tiet_san_pham.dart	100	Màn hình chi tiết mở bằng Navigator.push (NC2)
test/ (2 tệp)	52	5 bài kiểm thử: định dạng giá, dữ liệu mẫu, tổng tiền, smoke, thêm món
pubspec.yaml	25	Khai báo assets: - assets/images/

Thư mục nộp bài gồm lib/, pubspec.yaml và assets/; không kèm build/ và .dart_tool/. Thư mục docs/ chứa ảnh minh chứng và log thí nghiệm dùng cho báo cáo này.

Phần 1. Vì sao danh sách dài phải dùng ListView.builder? (≈ 15 phút)

ListView(children: [...]) nhận vào một danh sách widget đã được tạo sẵn. Nghĩa là ngay khi màn hình được dựng, Dart phải khởi tạo TẤT CẢ các widget con, đo và giữ chúng trong bộ nhớ — kể cả những dòng nằm ngoài màn hình mà người dùng chưa bao giờ nhìn tới. Với 6 sản phẩm thì không sao; với 10.000 sản phẩm thì thời gian mở màn hình tăng vọt và bộ nhớ bị chiếm vô ích.

ListView.builder thì ngược lại: ta chỉ đưa vào hai thứ là itemCount (có bao nhiêu dòng) và itemBuilder (hàm dựng dòng thứ index). Flutter chỉ gọi itemBuilder cho những dòng sắp lộ ra trong khung nhìn, rồi tái sử dụng lại khi người dùng cuộn. Đây chính là "lazy loading", và cũng là lý do giao diện vẫn mượt dù danh sách dài bao nhiêu.

Tiêu chí	ListView(children: [...])	ListView.builder
Thời điểm dựng widget	Dựng hết mọi dòng ngay khi mở màn hình	Chỉ dựng dòng đang/ sắp nhìn thấy
Bộ nhớ	Tỉ lệ với tổng số dòng	Gần như không đổi theo số dòng
Thời gian mở màn hình	Tăng nhanh theo số dòng	Gần như không đổi
Cuộn danh sách dài	Dễ giật, có thể hết bộ nhớ	Mượt, có tái sử dụng dòng
Cách viết	Liệt kê từng widget con	itemCount + itemBuilder(context, index)

```
// lib/screens/danh_sach_tab.dart – phần cốt lõi của tab Danh sách
return ListView.builder(
  padding: const EdgeInsets.fromLTRB(8, 4, 8, 92),
  itemCount: ds.length,          // có bao nhiêu dòng
  itemBuilder: dong,             // dòng thứ index trông như thế nào
);

Widget dong(BuildContext context, int index) {
  final sp = ds[index];
  debugPrint('F5_231A290134 itemBuilder dựng dòng $index – ${sp.ten}');
  return _dongSanPham(sp);
}
```

Khi cần đường kẻ giữa các dòng, chỉ việc đổi sang ListView.separated và thêm separatorBuilder — itemCount và itemBuilder giữ nguyên. Trong app, nút cạnh ô tìm kiếm bật/tắt qua lại giữa hai chế độ này để so sánh bằng mắt.

```
return ListView.separated(
  itemCount: ds.length,
  itemBuilder: dong,
  separatorBuilder: (_, _) => const Divider(
    height: 1, thickness: 1, indent: 16, endIndent: 16),
);
```

Phần 2. Model class — tách dữ liệu khỏi giao diện (≈ 15 phút)

Thay vì bốn mảng song song (tên[], mô tả[], giá[], ảnh[]), mỗi sản phẩm là một đối tượng SanPham. Cách này khiến trình biên dịch bắt lỗi gõ nhầm tên trường ngay lúc viết, và khi đổi nguồn dữ liệu sang API/ SQLite ở các lab sau thì phần giao diện không phải sửa dòng nào.

```
// lib/models/san_pham.dart
class SanPham {
  final int id;        // mã định danh duy nhất – dùng làm key cho Dismissible
  final String ten;
  final String moTa;
  final int gia;       // đơn vị đồng – dùng int cho tiền tệ, KHÔNG dùng double
  final String anh;    // ví dụ 'assets/images/p1.jpg'

  const SanPham({required this.id, required this.ten, required this.moTa,
                 required this.gia, required this.anh});
}
```

Trường	Kiểu	Ý nghĩa
id	int	Mã định danh duy nhất của sản phẩm; ổn định suốt vòng đời nên dùng làm key
ten	String	Tên sản phẩm, hiển thị đậm ở dòng đầu
moTa	String	Mô tả ngắn một dòng, cắt bằng ellipsis
gia	int	Giá tính bằng đồng; định dạng bằng dinhDangGia() khi hiển thị
anh	String	Đường dẫn asset, ví dụ assets/images/p1.jpg

Giá lưu bằng int (số đồng) chứ không dùng double: double có sai số làm tròn, cộng trừ nhiều lần sẽ ra kết quả lệch vài đồng — loại lỗi rất khó truy và rất khó giải thích với khách hàng. Việc định dạng "180000 → 180.000 đ" do hàm dinhDangGia() tự làm, không cần thư viện ngoài, nên có thể viết test cho nó.

```
// lib/data/kho_san_pham.dart
String dinhDangGia(int gia) {
  final s = gia.toString();
  final buf = StringBuffer();
  for (int i = 0; i < s.length; i++) {
    if (i > 0 && (s.length - i) % 3 == 0) buf.write('.');
    buf.write(s[i]);
  }
  return '${buf.toString()} đ';
}
```

Phần 3. Một dòng: Card + ListTile (≈ 20 phút)

Mỗi dòng là một Card bọc ListTile. ListTile chia sẵn bốn vùng, ta chỉ cần đặt đúng thứ vào từng vùng là có ngay bố cục gọn gàng, không phải tự tính khoảng cách bằng tay.

Vị trí	Nội dung thường đặt	Trong app F5
leading	Bên trái: ảnh, biểu tượng, avatar	Ảnh sản phẩm 56 × 56 bo góc 10, bọc trong AnhSanPham
title	Dòng chính	Tên sản phẩm, đậm, maxLines: 1
subtitle	Dòng phụ, chữ nhỏ hơn	Mô tả ngắn, maxLines: 1
trailing	Bên phải: giá, mũi tên, nút	Giá đã định dạng, đậm, màu primary

```
Card(
  margin: const EdgeInsets.symmetric(vertical: 4),
  clipBehavior: Clip.antiAlias,
  child: ListTile(
    onTap: () => _moChiTiet(sp),          // NC2: mở màn hình chi tiết
    leading: ClipRRect(
      borderRadius: BorderRadius.circular(10),
      child: AnhSanPham(duongDan: sp.anh, kichThuoc: 56),
    ),
    title: Text(sp.ten, maxLines: 1, overflow: TextOverflow.ellipsis,
                style: const TextStyle(fontWeight: FontWeight.w600)),
    subtitle: Text(sp.moTa, maxLines: 1, overflow: TextOverflow.ellipsis),
    trailing: Text(dinhDangGia(sp.gia), style: TextStyle(
                fontWeight: FontWeight.bold, color: theme.colorScheme.primary)),
  ),
)
```

Vì sao luôn phải có maxLines + overflow?

Tên sản phẩm do người dùng nhập, có thể dài tuỳ ý. Không đặt maxLines, một cái tên dài sẽ đẩy giá ra khỏi màn hình và sinh dải cảnh báo vàng đen “RenderFlex overflowed”. TextOverflow.ellipsis cắt bớt và thêm dấu ba chấm — đúng cách các ứng dụng thật vẫn làm.

Phần 4. Thêm / xóa bằng setState và Dismissible có key đúng (≈ 25 phút)

Danh sách hiển thị nằm trong một trường của State. Muốn Flutter vẽ lại sau khi dữ liệu đổi thì thao tác sửa dữ liệu bắt buộc phải nằm trong setState; nếu quên, dữ liệu đã đổi nhưng màn hình vẫn giữ nguyên ảnh cũ.

```
void _them() {
  setState(() {
    _demThem++;
    _tatCa.insert(0, SanPham(           // chèn lên ĐẦU danh sách
      id: _idKeTiep++,
      ten: 'Sản phẩm mới $_demThem',
      moTa: 'Vừa được thêm lúc $gio',
      gia: 50000 * _demThem,
      anh: 'assets/images/p1.jpg',
    ));
    _tuKhoa = '';                        // bỏ từ khóa để món mới luôn nhìn thấy
    _timKiem.clear();
  });
}
```

Với thao tác xóa, ngoài việc gọi setState ta còn phải nhớ vị trí cũ của phần tử để bài nâng cao NC3 khôi phục lại đúng chỗ khi người dùng bấm Hoàn tác.

```
void _xoa(SanPham sp) {
  final viTriGoc = _tatCa.indexWhere((e) => e.id == sp.id);
  if (viTriGoc < 0) return;
  setState(() => _tatCa.removeAt(viTriGoc));

  final messenger = ScaffoldMessenger.of(context);
  messenger.clearSnackBars();                    // chỉ giữ 1 SnackBar tại một thời điểm
  messenger.showSnackBar(SnackBar(
    duration: const Duration(seconds: 4),
    content: Text('Đã xóa ${sp.ten}'),
    action: SnackBarAction(label: 'Hoàn tác', onPressed: () {
      if (!mounted) return;
      setState(() {
        final viTri = viTriGoc > _tatCa.length ? _tatCa.length : viTriGoc;
        _tatCa.insert(viTri, sp);                // NC3: trả về đúng vị trí cũ
      });
    }),
  ));
}
```

4.1. Key của Dismissible — chi tiết quan trọng nhất của bài

```
return Dismissible(
  // key PHẢI duy nhất và ổn định → dùng id của dữ liệu, KHÔNG dùng index.
  key: ValueKey<int>(sp.id),
  direction: DismissDirection.endToStart,
  background: Container(... Colors.red.shade600 ..., child: Icon(Icons.delete)),
  onDismissed: (_) => _xoa(sp),
  child: Card(...),
);
```

Flutter so sánh cây widget cũ với cây widget mới để quyết định giữ lại hay dựng lại từng phần tử. Khi các phần tử có key riêng, Flutter ghép cặp theo key; khi không có key (hoặc key trùng), nó ghép theo vị trí. Với danh sách có thể xóa ở giữa, cách ghép theo vị trí là sai: xóa mục thứ 3 thì mục thứ 4 tụt lên vị trí 3 và bị coi là "mục vừa bị xóa". Dùng id làm key là cách duy nhất đúng, vì id không đổi khi danh sách co giãn. Thí nghiệm chứng minh điều này nằm ở Câu hỏi phân tích 1 (Phần 8).

Phần 5. Lưới với GridView.builder + gridDelegate (≈ 25 phút)

GridView.builder có cùng tinh thần với ListView.builder: itemCount + itemBuilder. Phần quyết định hình dáng lưới là gridDelegate — "người chia ô". Bài lab cần bản co giãn, nên dùng SliverGridDelegateWithMaxCrossAxisExtent.

Delegate	Cách chia ô	Nên dùng khi
SliverGridDelegateWithFixedCrossAxisCount	Cố định số cột	Đơn giản; nhưng trên tablet ô bị kéo rất to
SliverGridDelegateWithMaxCrossAxisExtent	Đặt bề rộng tối đa của một ô	Giao diện co giãn — Flutter tự tính số cột theo màn hình

```
// lib/screens/album_tab.dart
static const double maxCrossAxisExtent = 230;
static const double _spacing = 8;
static const double _paddingNgang = 16;   // 8 bên trái + 8 bên phải

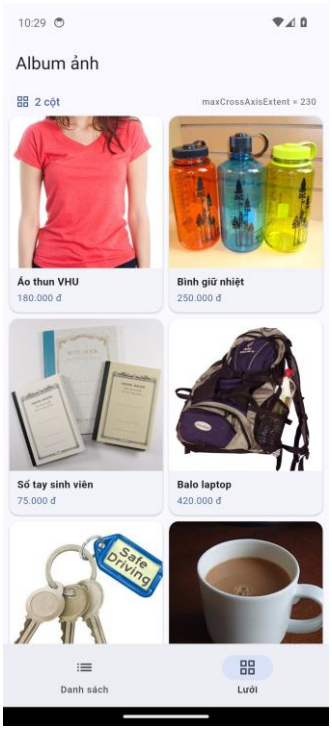
// Công thức y hệt cách delegate tính, để nhãn "n cột" luôn khớp với lưới thật:
final soCot = ((constraints.maxWidth - _paddingNgang) /
        (maxCrossAxisExtent + _spacing)).ceil();

GridView.builder(
  padding: const EdgeInsets.fromLTRB(8, 4, 8, 12),
  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
    maxCrossAxisExtent: maxCrossAxisExtent,
    crossAxisSpacing: _spacing,
    mainAxisSpacing: _spacing,
    childAspectRatio: 0.78,        // rộng / cao — càng nhỏ thì ô càng cao
  ),
  itemCount: duLieuMau.length,
  itemBuilder: (context, index) => _oLuoi(context, duLieuMau[index]),
)
```

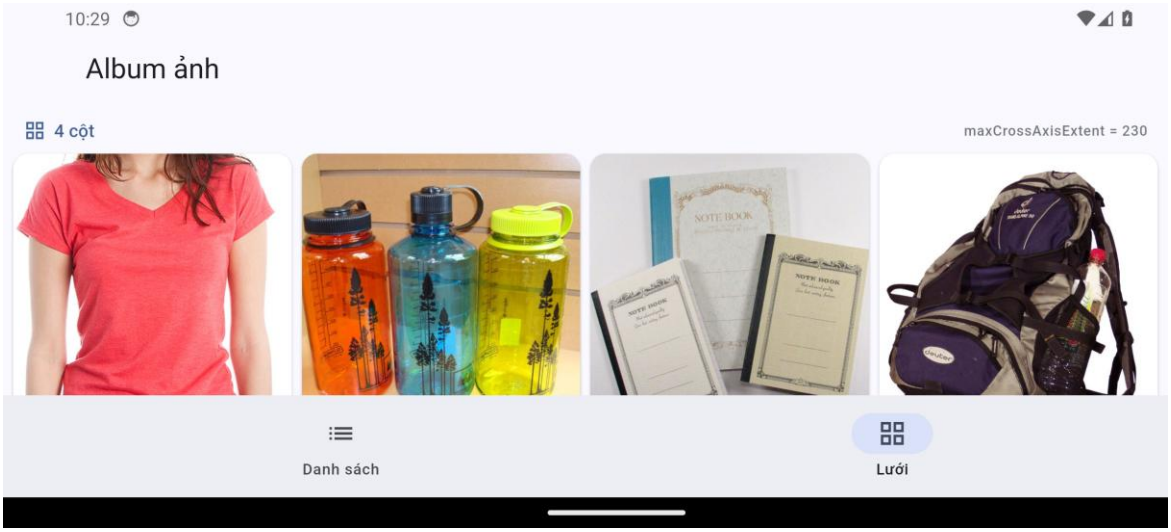
childAspectRatio — thủ phạm số một gây tràn ô

Ô lưới có kích thước do delegate quyết định, không co theo nội dung. childAspectRatio là rộng chia cao: số càng nhỏ thì ô càng cao. Nếu ảnh và hai dòng chữ cao hơn ô, Flutter hiện dải vàng đen. Cách xử lý trong bài: bọc ảnh trong Expanded (để ảnh nhận phần cao còn lại) và đặt maxLines: 1 cho tên sản phẩm, đồng thời chọn childAspectRatio 0.78 vừa đủ cho ảnh vuông và hai dòng chữ.

Trong app, dòng trên cùng của tab Lưới in ra số cột đang dùng và giá trị maxCrossAxisExtent, để khi xoay ngang có thể thấy ngay lưới tự chia lại. Hai ảnh dưới là cùng một đoạn mã, chỉ khác hướng màn hình.



(a) Màn hình dọc — 2 cột, ô vuông đều nhau, không tràn



(b) Xoay ngang — tự đổi thành 4 cột, ảnh không méo

Hình 2. NC4 — lưới tự đổi số cột theo bề rộng màn hình (maxCrossAxisExtent = 230).

Phần 6. Assets và Image.asset có errorBuilder (≈ 15 phút)

Muốn dùng ảnh trong app thì tệp ảnh phải nằm trong project VÀ phải được khai báo trong pubspec.yaml. Khai báo theo thư mục (có dấu "/" ở cuối) sẽ lấy mọi tệp trong thư mục đó, nhưng KHÔNG đệ quy xuống thư mục con — mỗi thư mục con phải có một dòng riêng. Sau khi sửa pubspec phải chạy flutter pub get (hoặc hot restart) thì app mới nhìn thấy ảnh.

```
# pubspec.yaml
flutter:
  uses-material-design: true
  assets:
    - assets/images/        # dấu / ở cuối là bắt buộc
```

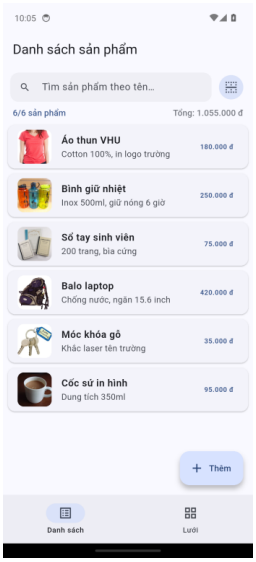
```
// lib/widgets/anh_san_pham.dart — dùng chung cho cả danh sách và lưới
Image.asset(
  duongDan,
  width: kichThuoc,
  height: kichThuoc,
  fit: BoxFit.cover,                  // ảnh cùng tỉ lệ vuông nên không bị méo
  errorBuilder: (context, error, stackTrace) => Container(
    color: Colors.grey.shade300,      // gõ sai đường dẫn vẫn ra ô xám, không nổ màn hình đỏ
    child: const Icon(Icons.image_not_supported, color: Colors.grey),
  ),
)
```

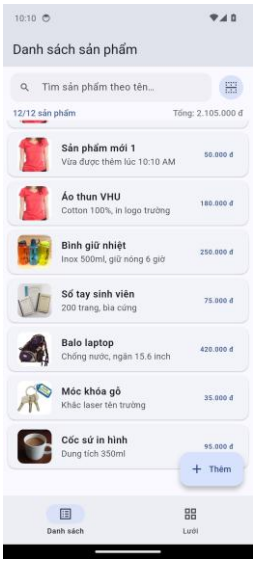
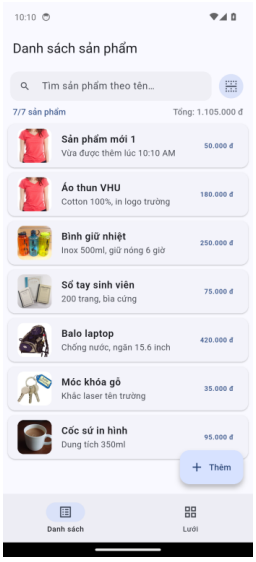
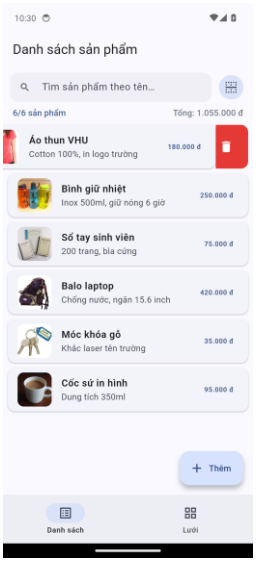
Sáu ảnh sản phẩm đều là ảnh vuông 600 × 600, dung lượng dưới 200 KB để cuộn không giật, và đều có giấy phép tự do (Wikimedia Commons / Wikipedia).

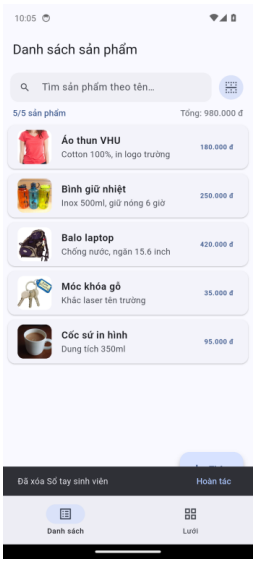
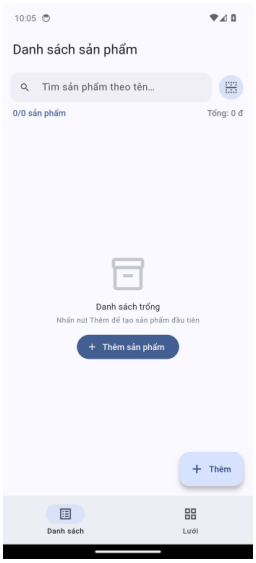
Mã	Sản phẩm	Tệp	Kích thước	Nguồn ảnh
1	Áo thun VHU	p1.jpg	600×600 · 59 KB	en.wikipedia.org/wiki/T-shirt
2	Bình giữ nhiệt	p2.jpg	600×600 · 65 KB	en.wikipedia.org/wiki/Water_bottle
3	Sổ tay sinh viên	p3.jpg	600×600 · 51 KB	en.wikipedia.org/wiki/Notebook
4	Balo laptop	p4.jpg	600×600 · 53 KB	en.wikipedia.org/wiki/Backpack
5	Móc khóa gỗ	p5.jpg	600×600 · 34 KB	en.wikipedia.org/wiki/Keychain
6	Cốc sứ in hình	p6.jpg	600×600 · 32 KB	en.wikipedia.org/wiki/Mug

Phần 7. Kiểm thử — bảng 7 kịch bản kèm ảnh minh chứng

Bảy kịch bản dưới đây đúng theo đề bài. Cột cuối là ảnh chụp màn hình thật lấy trực tiếp từ máy ảo Pixel 6 API 34 bằng adb (không dàn dựng lại).

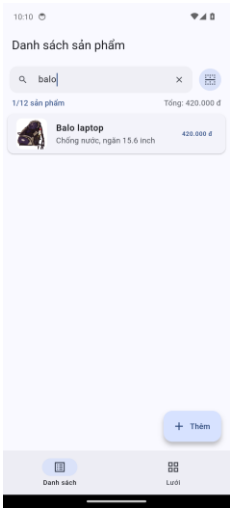
#	Thao tác	Kết quả đúng phải thấy	Ảnh minh chứng
1	Mở app ở tab Danh sách	6 sản phẩm hiện đủ ảnh, tên, mô tả và giá đã định dạng. Thực tế: “6/6 sản phẩm”, tổng 1.055.000 đ.	 *6/6 sản phẩm, tổng 1.055.000 đ*

#	Thao tác	Kết quả đúng phải thấy	Ảnh minh chứng
2	Cuộn nhanh lên xuống	Mượt, không giật, không có dải cảnh báo tràn. Thực tế: thêm tới 12 món rồi cuộn, log cho thấy chỉ vài dòng được dựng lại.	 *12/12 sản phẩm, cuộn không giật*
3	Bấm nút +	Sản phẩm mới chèn lên đầu danh sách. Thực tế: “Sản phẩm mới 1” nằm đầu, số đếm 6/6 → 7/7.	 *7/7 — mục mới ở đầu danh sách*
4	Vuốt một mục sang trái	Nền đỏ có biểu tượng thùng rác; thả ra thì mục biến mất. Thực tế: đúng như ảnh, thả ra SnackBar “Đã xóa …” hiện lên.	 *Nền đỏ + biểu tượng thùng rác khi đang vuốt*

#	Thao tác	Kết quả đúng phải thấy	Ảnh minh chứng
5	Vuốt xóa một mục ở giữa danh sách	Đúng mục đó biến mất (kiểm tra key). Thực tế: xóa "Sổ tay sinh viên" ở dòng 3, các mục còn lại giữ đúng thứ tự; bấm Hoàn tác thì quay lại 6/6.	 *Xóa đúng dòng 3 — SnackBar kèm Hoàn tác*
6	Xóa hết mọi mục	Hiện "Danh sách trống" thay vì màn hình trắng. Thực tế: 0/0 sản phẩm, có biểu tượng và nút "Thêm sản phẩm".	 *0/0 — "Danh sách trống"*
7	Chuyển tab Lưới, xoay ngang máy	Lưới vẫn các ô đều nhau, ảnh không méo, không tràn ô. Thực tế: dọc 2 cột, ngang 4 cột (đúng như công thức của delegate).	 *Xoay ngang — 4 cột, ảnh không méo*

Bảng 1. Bảy kịch bản kiểm thử theo đề bài, kèm ảnh chụp màn hình thật từ Pixel 6 API 34.

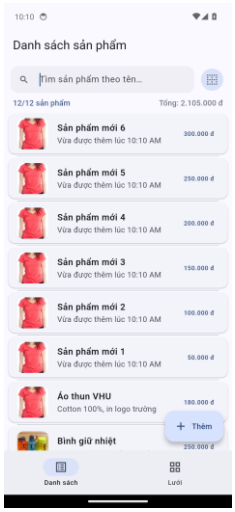
Ngoài bảy kịch bản bắt buộc, app còn được kiểm tra thêm ba trạng thái của các bài nâng cao: lọc theo tên (NC1, Hình 3a), màn hình chi tiết (NC2, Hình 3b) và chế độ ListView.separated (Hình 3c).



(a) NC1 — gõ "balo" lọc còn 1/12



(b) NC2 — chi tiết mở bằng Navigator.push



(c) ListView.separated — có đường kẻ

Hình 3. Ba trạng thái bổ sung: lọc theo tên (NC1), màn hình chi tiết (NC2), ListView.separated.

Phần 8. Ba câu hỏi phân tích

Câu 1. Đổi key thành ValueKey(index) rồi xóa một mục ở giữa — hiện tượng gì, vì sao?

Thí nghiệm được làm thật: tạm sửa key của Dismissible thành ValueKey<int>(index), build lại, mở app và vuốt xóa "Sổ tay sinh viên" (dòng thứ 3). Kết quả: dữ liệu vẫn đúng (số đếm còn 5/5, tổng tiền 980.000 đ) nhưng giao diện vỡ ngay — một vùng đỏ chiếm giữa màn hình kèm thông báo:

```
A dismissed Dismissible widget is still part of the tree.
Make sure to implement the onDismissed handler and to immediately remove
the Dismissible widget from the application once that handler has fired.
```



Hình 4. Lỗi thật khi dùng ValueKey(index): dải đỏ "A dismissed Dismissible widget is still part of the tree".

Giải thích: index của mọi mục phía sau thay đổi ngay khi xóa một mục ở giữa. Mục "Balo laptop" từ index 3 tụt xuống index 2, và Flutter dùng chính key 2 để ghép nó với phần tử cũ ở vị trí 2 — tức là ghép với Dismissible vừa bị xóa. Hệ quả là Dismissible đã hoàn tất vuốt xóa vẫn nằm lại trong cây widget, còn dòng dữ liệu mới thì chưa có widget riêng, nên Flutter báo lỗi và dừng vẽ phần còn lại của danh sách.

Log itemBuilder cũng cho thấy dữ liệu được dựng lại đúng 5 dòng (0 Áo thun, 1 Bình giữ nhiệt, 2 Balo, 3 Móc khóa, 4 Cốc sứ) — chứng tỏ lỗi nằm ở khâu ghép cặp widget, không phải ở dữ liệu:

```
F5_231A290134 itemBuilder dựng dòng 0 – Áo thun VHU
F5_231A290134 itemBuilder dựng dòng 1 – Bình giữ nhiệt
F5_231A290134 itemBuilder dựng dòng 2 – Balo laptop     <-- Sổ tay đã bị xóa khỏi dữ liệu
F5_231A290134 itemBuilder dựng dòng 3 – Móc khóa gỗ
F5_231A290134 itemBuilder dựng dòng 4 – Cốc sứ in hình
```

Kết luận: key của Dismissible phải vừa DUY NHẤT vừa ỔN ĐỊNH. Index không ổn định vì nó đổi theo vị trí trong danh sách; id của dữ liệu mới ổn định. Đó là lý do bản chính thức dùng ValueKey<int>(sp.id). Sau thí nghiệm, mã đã được trả lại như cũ (ảnh Hình 1c và Bảng 1 kịch bản 5 là kết quả của bản chính thức).

Câu 2. Đổi sang maxCrossAxisExtent: 180 thì số cột khi xoay ngang thay đổi thế nào?

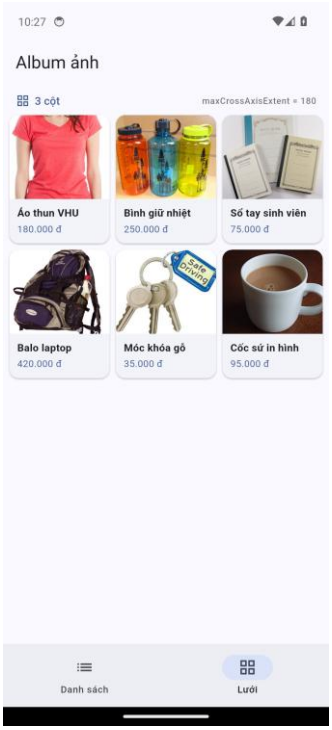
Đây là thí nghiệm thật, không suy đoán: sửa hằng số thành 180, build lại và chụp cả hai hướng màn hình. Kết quả: màn hình dọc từ 2 cột thành 3 cột; xoay ngang từ 4 cột thành 5 cột.

Hướng màn hình	Bề rộng khả dụng	maxCrossAxisExtent = 230	maxCrossAxisExtent = 180
Dọc (1080 px)	1080 / 2,625 − 16 ≈ 395,4 dp	ceil(395,4 / 238) = 2 cột	ceil(395,4 / 188) = 3 cột
Ngang (2400 px)	2400 / 2,625 − 16 ≈ 898,3 dp	ceil(898,3 / 238) = 4 cột	ceil(898,3 / 188) = 5 cột

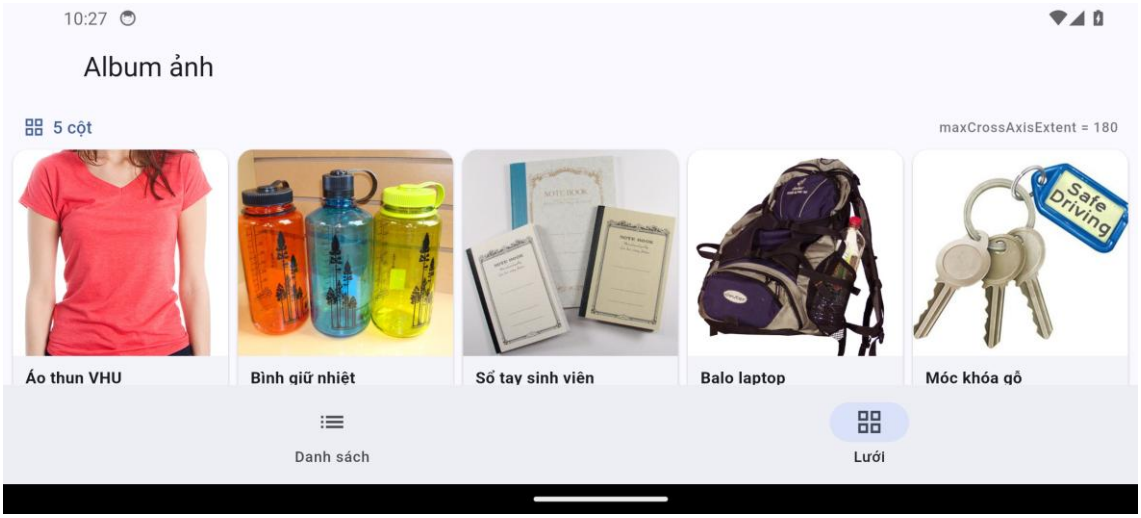
Vì sao lại chia cho 238 và 188 mà không phải 230 và 180? SliverGridDelegateWithMaxCrossAxisExtent tính số cột bằng công thức:

```
số cột = ceil( bề rộng còn lại / (maxCrossAxisExtent + crossAxisSpacing) )
       = ceil( 898,3 / (180 + 8) ) = ceil(4,78) = 5
```

Tức là bề rộng tối đa phải cộng thêm khoảng cách giữa các ô, vì khoảng cách đó cũng chiếm chỗ trên hàng. Vì vậy giảm maxCrossAxisExtent thì ô nhỏ lại và số cột tăng: ngang màn hình nhiều cột hơn nên mỗi ô hẹp hơn, ảnh và chữ trong ô cũng nhỏ hơn — phù hợp khi muốn xem nhiều ảnh cùng lúc, còn muốn ô to dễ bấm thì tăng maxCrossAxisExtent (ví dụ 300 sẽ cho 1 cột ở màn hình dọc).



(a) Dọc — 3 cột (thay vì 2 cột)



(b) Ngang — 5 cột (thay vì 4 cột)

Hình 5. Thí nghiệm maxCrossAxisExtent = 180: màn hình dọc 3 cột, xoay ngang 5 cột.

Câu 3. ListView.builder có gọi itemBuilder cho mọi phần tử ngay khi mở màn hình không?

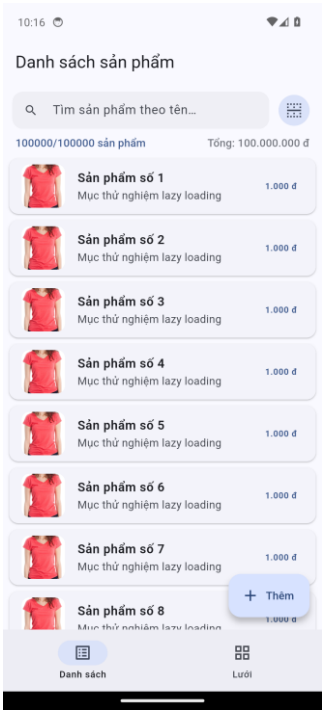
Không. itemBuilder chỉ được gọi cho những dòng nằm trong khung nhìn (cộng thêm một vùng đệm nhỏ mà Flutter chuẩn bị sẵn để cuộn mượt). Muốn chứng minh, chỉ cần thêm một dòng print vào itemBuilder — trong app đã có sẵn dòng đó:

```
debugPrint('F5_231A290134 itemBuilder dựng dòng $index – ${sp.ten}');
```

Thí nghiệm: tạm sinh 100.000 sản phẩm trong danh sách rồi mở màn hình. Số đếm ở đầu danh sách hiện "100000/100000 sản phẩm", nhưng bộ lọc logcat theo chữ "itemBuilder" chỉ ra đúng 11 dòng:

```
F5_231A290134 itemBuilder dựng dòng 0 – Sản phẩm số 1
F5_231A290134 itemBuilder dựng dòng 1 – Sản phẩm số 2
…
F5_231A290134 itemBuilder dựng dòng 10 – Sản phẩm số 11

→ 11 / 100000 dòng được dựng. Danh sách vẫn mở ra tức thì và cuộn bình thường.
```



Hình 6. Thí nghiệm lazy loading: "100000/100000 sản phẩm" nhưng chỉ 11 dòng được itemBuilder dựng (log đầy đủ ở docs/logs/lazy_100000.txt).

Đối chiếu: nếu thay bằng ListView(children: [...]), Flutter buộc phải tạo đủ 100.000 widget ngay khi mở màn hình. Số lần dựng widget sẽ bằng đúng số phần tử, thời gian mở màn hình tăng theo số phần tử và bộ nhớ phình lên tương ứng — đó chính là lý do danh sách dài phải dùng builder. Sau thí nghiệm, mã đã được trả lại danh sách 6 sản phẩm.

Phần 9. Đối chiếu với Lab A9 / A10 (Android)

Việc cần làm	Android (Java)	Flutter (Dart)
Hiển thị danh sách dài	RecyclerView + LayoutManager	ListView.builder
Mô tả một dòng	Lớp Adapter + ViewHolder + layout item	Hàm itemBuilder trả về widget
Số phần tử	getItemCount()	itemCount
Gán dữ liệu cho một dòng	onBindViewHolder(holder, position)	itemBuilder(context, index)
Báo dữ liệu đã đổi	notifyItemInserted / notifyDataSetChanged	setState
Lưới	GridLayoutManager	GridView.builder + gridDelegate
Vuốt để xóa	ItemTouchHelper	Dismissible
Trạng thái rỗng	setEmptyView() hoặc tự ẩn/hiện View	if (ds.isEmpty) return widget trạng thái rỗng

Nhận xét: Flutter gộp Adapter + ViewHolder + layout item thành một hàm duy nhất nên viết ít hơn hẳn. Ở Lab A9, một dòng món ăn cần bốn phần: lớp MonAnAdapter, lớp lồng ViewHolder, item_mon_an.xml và phương thức getView; ở F5 chỉ cần hàm itemBuilder trả về một Card +

ListTile. Ngược lại, Android tách rõ từng phần nên trong dự án lớn việc phân công và kiểm soát dễ hơn, đồng thời RecyclerView có sẵn nhiều tiện ích như ItemTouchHelper hay DiffUtil mà bên Flutter phải tự làm hoặc dùng thư viện ngoài.

Điểm giống nhau về bản chất: cả hai đều là "danh sách lười" — chỉ tạo đủ dòng cho màn hình rồi tái sử dụng. Ở A9, việc tái sử dụng diễn ra qua tham số convertView và setTag/getTag; ở F5, Flutter lo phần đó giúp ta, còn ta chỉ cần khai báo itemCount và itemBuilder.

Phần 10. Bài nâng cao (đề yêu cầu ít nhất 1 — đã làm cả 4)

Cả bốn bài nâng cao đều đã được cài đặt và chạy thật trên máy ảo.

NC1. Thanh tìm kiếm lọc danh sách theo tên khi đang gõ

Ô tìm kiếm dùng TextField(onChanged: …) rồi lọc bằng where(...).toList(). Danh sách đang hiển thị (biến ds) là danh sách đã lọc, còn _tatCa là dữ liệu gốc — nhờ vậy dòng đếm "x/y sản phẩm" luôn cho biết đang xem bao nhiêu trên tổng bao nhiêu.

```
List<SanPham> get _hienThi {
  final k = _tuKhoa.trim().toLowerCase();
  if (k.isEmpty) return List.of(_tatCa);
  return _tatCa.where((sp) => sp.ten.toLowerCase().contains(k)).toList();
}
```

Bằng chứng: gõ "balo" → còn 1/12 sản phẩm, tổng tiền tự cập nhật thành 420.000 đ (Hình 3a). Khi lọc không ra kết quả, màn hình hiện "Không tìm thấy sản phẩm phù hợp" kèm nút xóa từ khóa.

NC2. Bấm một mục thì mở màn hình chi tiết

Cả dòng trong danh sách lẫn ô trong lưới đều mở được màn hình chi tiết bằng Navigator.push. Màn hình này nhận nguyên đối tượng SanPham nên không cần truy vấn lại dữ liệu.

```
Navigator.of(context).push(MaterialPageRoute<void>(
  builder: (_) => ChiTietSanPhamScreen(sanPham: sp),
));
```

Chi tiết hiển thị ảnh lớn bo góc, tên, chip giá, chip mã sản phẩm, mô tả và bảng thông tin đầy đủ (tên / giá / đường dẫn ảnh) — xem Hình 3b.

NC3. SnackBar "Hoàn tác" sau khi xóa

Trước khi xóa, app ghi lại vị trí của phần tử. SnackBar hiện trong 4 giây với nút Hoàn tác; bấm vào thì chèn lại đúng vị trí cũ. Để tránh nhiều SnackBar xếp hàng chồng lên nhau, app gọi clearSnackBars() trước khi hiện cái mới, và đặt thêm một Future.delayed(4s) để tự ẩn — nhờ vậy khi xóa liên tiếp nhiều mục, SnackBar không "đè" lên màn hình trạng thái rỗng.

Bằng chứng: xóa "Sổ tay sinh viên" → SnackBar "Đã xóa Sổ tay sinh viên" + nút Hoàn tác (Hình 1c) → bấm Hoàn tác → danh sách trở lại 6/6 đúng thứ tự ban đầu (docs/screenshots/f5_undo.png).

NC4. Lưới tự đổi số cột theo bề rộng màn hình

Đã trình bày ở Phần 5: dùng SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent: 230) nên màn hình dọc 2 cột, xoay ngang 4 cột, không cần viết if theo hướng màn hình. Nhãn số cột ở đầu tab được tính bằng đúng công thức của delegate nên luôn khớp với lưới thật (xem Hình 2 và Câu hỏi phân tích 2).

Phần 11. Năm câu hỏi ôn tập

1. ListView.builder khác ListView(children: [...]) ở điểm nào?

ListView(children: [...]) nhận danh sách widget đã dựng sẵn, nên mọi phần tử được tạo ngay khi màn hình mở — tốn thời gian và bộ nhớ theo số phần tử. ListView.builder nhận itemCount và itemBuilder, chỉ gọi itemBuilder cho những dòng đang hoặc sắp nhìn thấy rồi tái sử dụng khi cuộn. Nhờ vậy danh sách rất dài vẫn mở nhanh và cuộn mượt. Trong thí nghiệm 100.000 mục, builder chỉ dựng 11 dòng.

2. itemCount và itemBuilder làm nhiệm vụ gì?

itemCount trả lời câu hỏi "danh sách có bao nhiêu dòng?" — thiếu hoặc sai nó thì hoặc không có dòng nào, hoặc truy cập quá chỉ số gây lỗi RangeError. itemBuilder(context, index) trả lời "dòng thứ index trông như thế nào?" — nhận chỉ số và trả về một widget. Flutter gọi hai thứ này theo nhu cầu vẽ, không gọi một lần cho toàn bộ danh sách.

3. Vì sao Dismissible bắt buộc phải có key duy nhất và ổn định?

Flutter so sánh cây widget cũ và mới để biết phần tử nào giữ lại, phần tử nào dựng lại. Có key thì ghép cặp theo key; không có key thì ghép theo vị trí. Với danh sách xóa được ở giữa, ghép theo vị trí là sai: xóa mục thứ 3 thì mục thứ 4 tụt lên và bị coi là mục vừa xóa. Dùng ValueKey(index) cũng sai vì index thay đổi ngay khi xóa — thí nghiệm ở Câu hỏi phân tích 1 đã tạo ra lỗi "A dismissed Dismissible widget is still part of the tree". Vì vậy key phải lấy từ dữ liệu, ví dụ ValueKey<int>(sp.id).

4. childAspectRatio ảnh hưởng thế nào tới ô lưới?

childAspectRatio là tỉ lệ rộng chia cao của mỗi ô. Số càng nhỏ thì ô càng cao (và ngược lại). Ô lưới có kích thước do delegate quyết định chứ không co theo nội dung, nên nếu nội dung cao hơn ô sẽ xuất hiện dải cảnh báo vàng đen. Trong bài, chọn 0.78 kèm bọc ảnh trong Expanded và đặt maxLines: 1 cho tên sản phẩm để ô vừa khít nội dung, không tràn.

5. Sau khi thêm ảnh mới vào assets/, phải làm những bước gì để app nhìn thấy ảnh đó?

Bốn bước: (1) chép tệp ảnh vào đúng thư mục, ví dụ assets/images/; (2) khai báo thư mục đó trong pubspec.yaml dưới khoá flutter: assets: với dấu "/" ở cuối (lưu ý khai báo thư mục không đệ quy, thư mục con phải khai riêng); (3) chạy flutter pub get để cập nhật gói tài nguyên; (4) hot restart (không phải hot reload) hoặc chạy lại app, vì danh sách asset được chốt lúc khởi động. Nếu gõ sai đường dẫn, errorBuilder sẽ hiện ô xám thay vì làm app lỗi.

Phần 12. Kết luận và hướng phát triển

Bài lab đã hoàn thành đủ các yêu cầu: dùng ListView.builder đúng itemCount/itemBuilder, biết thêm ListView.separated khi cần đường kẻ; tách model class ra khỏi giao diện; dựng dòng bằng Card + ListTile có maxLines và overflow; thêm và xóa bằng setState với Dismissible mang key duy nhất lấy từ id dữ liệu; dựng lưới bằng GridView.builder + gridDelegate và hiểu vai trò của childAspectRatio; khai báo assets đúng trong pubspec.yaml và hiển thị ảnh bằng Image.asset có errorBuilder. Ngoài ra đã làm cả bốn bài nâng cao NC1–NC4.

Ba thí nghiệm trong báo cáo đều được thực hiện thật trên máy ảo: ValueKey(index) để thấy lỗi ghép cặp widget, đổi maxCrossAxisExtent để thấy số cột thay đổi, và danh sách 100.000 mục để thấy builder chỉ dựng 11 dòng. Nhờ vậy phần giải thích dựa trên quan sát chứ không phải suy đoán.

Hướng phát triển tiếp theo: (1) thay duLieuMau bằng dữ liệu từ REST API hoặc SQLite — phần giao diện không phải sửa vì dữ liệu đã tách khỏi model; (2) dùng AnimatedList hoặc SliverAnimatedList để có hiệu ứng thêm/xóa mượt hơn; (3) thêm tìm kiếm có debounce và sắp xếp theo giá; (4) dùng GridView.builder với SliverGridDelegateWithFixedCrossAxisCount trên tablet nếu muốn số cột cố định; (5) bổ sung ảnh lớn trong màn hình chi tiết kèm Hero animation để chuyển cảnh liền mạch.

Phụ lục A. Kết quả flutter analyze và flutter test

Lệnh bắt buộc trước khi nộp là flutter analyze; kết quả trên project này:

```
> flutter analyze
Analyzing F5_231A290134...
No issues found! (ran in 2.9s)
```

Bộ kiểm thử tự động gồm 5 bài, tất cả đều đạt:

```
> flutter test
00:00 +0: dinhDangGia chèn dấu chấm phân cách nghìn
00:00 +1: dữ liệu mẫu có 6 sản phẩm với id duy nhất
00:00 +2: tổng tiền của dữ liệu mẫu
00:01 +3: mở app hiện 2 tab và danh sách 6 sản phẩm
00:02 +4: bấm Thêm thì sản phẩm mới nằm ở đầu danh sách
00:02 +5: All tests passed!
```

Phụ lục B. Tự đánh giá theo thang điểm của đề

Tiêu chí	Điểm tối đa	Kết quả trong bài
ListView.builder chạy đúng với itemCount / itemBuilder	2,0	2,0 — tab Danh sách dùng builder; có thêm ListView.separated
Model class riêng, dữ liệu tách khỏi giao diện	1,0	1,0 — SanPham + kho_san_pham.dart, giao diện không giữ dữ liệu gốc
Card + ListTile gọn, có maxLines và overflow	1,5	1,5 — title/subtitle đều maxLines: 1 + ellipsis
Thêm/xóa bằng setState, Dismissible có key đúng	2,0	2,0 — ValueKey<int>(sp.id); có thí nghiệm chứng minh vì sao không dùng index

Tiêu chí	Điểm tối đa	Kết quả trong bài
GridView.builder không tràn ô, assets khai báo đúng	1,5	1,5 — 2 cột dọc / 4 cột ngang, không có dải vàng đen; assets khai báo theo thư mục
Bảng 7 kịch bản + bảng đối chiếu A9/A10	1,0	1,0 — Phần 7 (bảng có ảnh) và Phần 9
Bài nâng cao	1,0	1,0 — làm cả 4 bài NC1–NC4
Tổng	10,0	10,0

Phụ lục C. Danh mục ảnh minh chứng

Tệp	Nội dung
f5_home.png	Kịch bản 1 — tab Danh sách, 6/6 sản phẩm, tổng 1.055.000 đ
f5_scroll.png	Kịch bản 2 — 12/12 sản phẩm sau khi cuộn danh sách
f5_add.png	Kịch bản 3 — bấm +, "Sản phẩm mới 1" ở đầu danh sách, 7/7
f5_swipe.png	Kịch bản 4 — đang vuốt: nền đỏ và biểu tượng thùng rác
f5_delete.png	Kịch bản 5 — xóa đúng "Sổ tay sinh viên" ở giữa, SnackBar có Hoàn tác
f5_undo.png	NC3 — sau khi bấm Hoàn tác, danh sách trở lại 6/6
f5_empty.png	Kịch bản 6 — xóa hết, hiện "Danh sách trống"
f5_grid.png	Kịch bản 7 (dọc) — lưới 2 cột
f5_landscape.png	Kịch bản 7 (ngang) — lưới 4 cột
f5_search.png	NC1 — lọc "balo" còn 1/12 sản phẩm
f5_detail.png	NC2 — màn hình chi tiết mở bằng Navigator.push
f5_separated.png	ListView.separated — có đường kẻ giữa các dòng
f5_key_index_error.png	Câu hỏi phân tích 1 — lỗi khi dùng ValueKey(index)
f5_lazy_100k.png	Câu hỏi phân tích 3 — 100.000 mục, chỉ 11 dòng được dựng
f5_grid_180_doc.png	Câu hỏi phân tích 2 — maxCrossAxisExtent 180, màn hình dọc 3 cột
f5_grid_180_ngang.png	Câu hỏi phân tích 2 — maxCrossAxisExtent 180, xoay ngang 5 cột

Log thí nghiệm được lưu tại docs/logs/: valuekey_index_rebuild.txt (log itemBuilder sau khi xóa mục giữa bằng ValueKey(index)) và lazy_100000.txt (toàn bộ 11 dòng log của thí nghiệm 100.000 mục).